

Backend API Tasarımı Notları

(Pratik ve Üretim Odaklı Bakış)

Giriş

Bu doküman, bir backend geliştiricisinin *API tasarımı* konusunda bilmesi gereken temel ve ileri seviyedeki kavramları pratik bir bakışla özetler. Odak noktası:

- HTTP ve REST temelleri,
- Kaynak (resource) tasarımı,
- Request/response sözleşmeleri,
- Validasyon ve hata yönetimi,
- Kimlik doğrulama ve yetkilendirme,
- Performans, güvenlik ve gözlemlenebilirlik,
- API evrimi, versiyonlama ve geliştirici deneyimi (DX).

Amaç, hem günlük geliştirme işinde kullanabileceğin, hem de uzun vadede *senior seviye* API tasarım perspektifi kazandıracak bir referans oluşturmaktır.

HTTP ve REST Temelleri

HTTP Metodları

En yaygın HTTP metodları ve tipik kullanım senaryoları:

- **GET:** Veri okuma. Yan etkisiz ve idempotent olmalıdır.
- **POST:** Yeni kaynak oluşturma veya yan etkili bir işlem başlatma.
- **PUT:** Mevcut kaynağı *tümüyle* güncelleme (replace semantics).
- **PATCH:** Mevcut kaynağı kısmi olarak güncelleme.
- **DELETE:** Kaynak silme. İdempotent olması tercih edilir.

HTTP Durum Kodları

Temel status kod grupları (200–500 aralığındaki yaygın kodlar):

- **2xx – Başarılı (Success)**
 - 200 OK – İstek başarıyla işlendi, gövde genelde yanıtı içerir.
 - 201 Created – Yeni bir kaynak başarıyla oluşturuldu (Location header kullanılabilir).
 - 202 Accepted – İstek kabul edildi, ancak işlem asenkron olarak daha sonra tamamlanacak.
 - 203 Non-Authoritative Information – Cevap, orijinal sunucudan farklı bir kaynaktan gelmiş olabilir.
 - 204 No Content – İstek başarıyla işlendi, dönecek body yok (ör. silme, basit güncelleme).
 - 205 Reset Content – İstek sonrası istemcinin form vb. arayüzü sıfırlaması önerilir.
 - 206 Partial Content – İçeriğin sadece bir kısmı döndürülüyor (Range istekleri).
- **3xx – Yönlendirme (Redirection)**
 - 300 Multiple Choices – Birden fazla olası kaynak/temsili mevcut.
 - 301 Moved Permanently – Kaynak kalıcı olarak yeni bir URI'ye taşındı.
 - 302 Found – Geçici yönlendirme (tarihsel olarak farklı yorumlanabiliyor).
 - 303 See Other – Cevap başka bir URI'de, genelde POST sonrası GET'e yönlendirme.
 - 304 Not Modified – İçerik değişmedi; istemci cache'lenmiş kopyayı kullanabilir.
 - 307 Temporary Redirect – Geçici yönlendirme, HTTP metodunu değiştirmez.
 - 308 Permanent Redirect – Kalıcı yönlendirme, HTTP metodunu değiştirmez.
- **4xx – İstemci Hatası (Client Error)**
 - 400 Bad Request – Geçersiz istek (parse hatası, eksik/zayıf input vb.).
 - 401 Unauthorized – Kimlik doğrulaması gerekiyor veya token geçersiz.
 - 402 Payment Required – Rezerv; bazı API'ler ödeme/limit aşımalarında kullanır.
 - 403 Forbidden – Kimlik doğrulansa bile bu kaynağa erişim yetkisi yok.
 - 404 Not Found – İstenen kaynak bulunamadı.
 - 405 Method Not Allowed – Endpoint var ama kullanılan HTTP metodu desteklenmiyor.

- 406 Not Acceptable – İstemcinin Accept header’ına uygun içerik üretilemedi.
- 407 Proxy Authentication Required – Proxy üzerinden kimlik doğrulama gerekli.
- 408 Request Timeout – Sunucu, istemciden tam isteği zamanında alamadı.
- 409 Conflict – Kaynak durumu ile çelişen bir istek (ör. versiyon çakışması, duplicate).
- 410 Gone – Kaynak önceden vardı ama artık kalıcı olarak yok.
- 412 Precondition Failed – If-Match/If-None-Match gibi ön koşullar sağlanmadı.
- 413 Payload Too Large – Gönderilen gövde sunucunun kabul ettiğinden büyük.
- 414 URI Too Long – İstek URI’si sunucunun işlemek için çok uzun.
- 415 Unsupported Media Type – Gönderilen içerik tipi desteklenmiyor.
- 416 Range Not Satisfiable – İstenen byte aralığı içerikte mevcut değil.
- 422 Unprocessable Entity – Söz dizimi doğru ama semantik/validasyon hatası var.
- 429 Too Many Requests – Rate limit aşıldı, bir süre sonra tekrar denenmeli.
- 431 Request Header Fields Too Large – Header’lar çok büyük veya çok fazla.
- 451 Unavailable For Legal Reasons – Hukuki sebeplerle içerik sunulmuyor.

- **5xx – Sunucu Hatası (Server Error)**

- 500 Internal Server Error – Beklenmeyen genel sunucu hatası.
- 501 Not Implemented – İstenen metod/fonksiyonellik sunucuda uygulanmamış.
- 502 Bad Gateway – Proxy/gateway, üst servisten geçerli bir cevap alamadı.
- 503 Service Unavailable – Sunucu geçici olarak kullanılamıyor (bakım, aşırı yük).
- 504 Gateway Timeout – Proxy/gateway, üst servisten zamanında yanıt alamadı.
- 505 HTTP Version Not Supported – İstenen HTTP sürümü desteklenmiyor.

Resource Odaklı URL Tasarımı

REST yaklaşımında URL’ler *isim tabanlı* ve kaynak odaklıdır:

- /users
- /users/{id}
- /users/{id}/subscriptions

Resource Odaklı URL Tasarımı ve Fiillerden Kaçınma

REST mimarisinde temel prensiplerden biri, **URL yolunun** bir *resource*'u (kaynağı) temsil etmesi, **HTTP metodunun** ise bu kaynak üzerinde yapılan *eylemi* (fiili) ifade etmesidir. Bu nedenle, endpoint'lerin path kısmında fiil (*create, get, update* vb.) kullanmaktan kaçınılır.

URL = Kaynak (İsim), HTTP Metodu = Eylem (Fiil)

Genel kural şu şekilde özetlenebilir:

- **URL (path):** “*Hangi şey?*” sorusunun cevabı, yani **isim** (ör. `users`, `orders`).
- **HTTP metodu:** “*Bu şeyle ne yapıyorum?*” sorusunun cevabı, yani **fiil** (GET, POST, PATCH, DELETE).

Bu bakış açısıyla:

- Kötü örnek: POST `/createUser`
- İyi örnek: POST `/users`

Yukarıdaki iyi örnekte:

- `/users` yolu, “kullanıcılar” isimli resource'u temsil eder.
- POST metodu, bu listeye yeni bir eleman ekleme (yeni kullanıcı oluşturma) fiilini ifade eder.

Böylece eylemi hem path içinde (`createUser`) hem de HTTP metodunda (POST) *iki kez* tekrar etmek yerine, eylemi sadece HTTP metoduna, kaynağı ise sadece URL'ye yüklemiş oluruz.

Aynı Resource İçin CRUD Operasyonlarının Gösterimi

Fiil içeren endpoint'ler yerine, tek bir resource ismi etrafında HTTP metodlarının kullanılması, CRUD işlemlerini son derece tutarlı hale getirir. Örneğin `users` resource'u için:

GET	<code>/users</code>	-> tüm kullanıcıları listele
POST	<code>/users</code>	-> yeni bir kullanıcı oluştur
GET	<code>/users/123</code>	-> id'si 123 olan kullanıcıyı getir
PATCH	<code>/users/123</code>	-> 123 numaralı kullanıcının bazı alanlarını güncelle
PUT	<code>/users/123</code>	-> 123 numaralı kullanıcıyı tamamen güncelle/replace et
DELETE	<code>/users/123</code>	-> 123 numaralı kullanıcıyı sil

Bu tasarımda hiçbir path, `/getUser`, `/createUser`, `/updateUser`, `/deleteUser` gibi fiil içeren isimler kullanmaz; çünkü bu fiiller zaten sırasıyla GET, POST, PATCH/PUT, DELETE metodlarıyla ifade edilmektedir.

Fiilli Endpoint'lerin Sakıncaları

Path içinde fiil kullanmak, pratikte şu sorunlara yol açabilir:

1. Tutarsız isimlendirme:

- `/getUsers`, `/createUser`, `/deleteUser` gibi farklı kalıplar, aslında aynı resource'a (`user`) ait eylemleri temsil eder.
- Bu durum, API yüzeyini karmaşık ve tahmin edilmesi zor hale getirir.

2. HTTP metodunun anlamını boşa çıkarmak:

- Bazı projelerde `/getUsers` gibi path'ler bile `POST` ile çağrılabilmekte, bu da “okuma GET ile yapılır” prensibini bozar.
- Bu durumda HTTP metodları, semantik anlamını kaybeder ve sadece “rasgele seçilmiş” bir taşıyıcıya dönüşür.

3. Araç ve altyapı desteğini kaybetmek:

- Birçok cache, proxy, gateway veya framework, `GET /users` gibi *standart* REST kalıplarını tanır ve onlar için optimize edilmiş davranışlar sunar.
- `POST /getUsers` gibi alışılmadık kalıplar kullanıldığında bu otomatik optimizasyonlardan yararlanmak zorlaşır.

Ne Zaman Fiil Kullanılabilir? Özel Aksiyon Endpoint'leri

Her ne kadar genel kural “path'te fiil kullanma” olsa da, bazı durumlarda resource üzerinde *özel bir aksiyon* tanımlamak gerekebilir. Örneğin bir siparişi iptal etmek:

`POST /orders/123/cancel`

Burada:

- `/orders/123` bir resource örneğini (id'si 123 olan siparişi),
 - `/cancel` ise bu resource üzerinde gerçekleştirilen özel bir eylemi temsil eder.
- `POST` metodu, bu aksiyonun tetiklenmesi için kullanılır.

Bu tür durumlarda fiil, bağımsız bir “ana resource” ismi olarak değil, zaten var olan bir resource'un altındaki *eylem* olarak kullanılır. Dolayısıyla `/cancelOrder` gibi başlı başına bir resource yaratmak yerine, “sipariş 123'ü iptal et” anlamını `/orders/123/cancel` gibi bir alt path ile ifade etmek tercih edilir.

Resource Tasarımı

Domain Odaklı Düşünme

API tasarımına başlarken doğrudan veritabanı tablolarından değil, *domain kavramlarından* yola çıkmak önemlidir:

- Kullanıcı (User),
- Abonelik (Subscription),
- Ödeme (Payment),
- Sipariş (Order),
- Randevu (Appointment) vb.

Her resource için:

- Benzersiz kimlik (ID) stratejisi (integer / UUID),
- Zorunlu ve opsiyonel alanlar,
- Diğer kaynaklarla ilişkiler (ör: bir kullanıcının randevuları).

Domain Odaklı Düşünmeyi Nasıl Uygularız?

Sadece veritabanı tablolarından yola çıkmak yerine, önce çözdüğümüz iş problemini (domain'i) anlamaya odaklanmak gerekir. Bu yaklaşım, “**önce iş kuralları, sonra veri modeli**” ilkesine dayanır.

Aşağıdaki adımlar, domain odaklı düşünmeyi uygulamak için pratik bir rehber sunar.

Kullanım Senaryolarını (User Story) Yazmak

Önce, sistemin gerçek hayatta nasıl kullanılacağını doğal dilde ifade ederiz. Örneğin bir randevu sistemi için:

“Hasta uygulamaya girer, uygun bir doktordan uygun bir saat seçer ve randevu alır. Randevudan 24 saat önce hatırlatma gönderilir. Hasta, randevudan 2 saat öncesine kadar ceza almadan randevuyu iptal edebilir.”

Bu tür cümleler, domain'deki aktörleri ve süreçleri ortaya çıkarır.

İsimleri (Noun) ve Fiilleri (Verb) Ayıklamak

Yukarıdaki anlatımdan, öncelikle geçen *isimleri* ve *fiilleri* çıkarırız:

- **İsimler (aday domain kavramları):** Hasta (Patient/User), Doktor (Doctor), Randevu (Appointment), Zaman Aralığı (TimeSlot), Hatırlatma (Reminder), Cezai yaptırım (Penalty/Policy) vb.
- **Fiiller (eylemler):** randevu almak (book), iptal etmek (cancel), hatırlatma göndermek (send reminder), uygunluk kontrol etmek (check availability) vb.

Bu aşamada henüz tablo tasarlamıyor, sadece iş dünyasındaki *nesneleri* ve *eylemleri* tanımlıyoruz.

Her Domain Nesnesinin Sorumluluğunu Belirlemek

Seçtiğimiz her domain kavramı için şu soruları sorarız (örneğin **Appointment** için):

- Hangi bilgileri taşımali? (ör. **patientId**, **doctorId**, **startAt**, **endAt**, **status**, **createdAt** vb.)
- Hangi iş kuralları geçerli? (ör. geçmiş zamana randevu alınamaz; randevu saatine 2 saatten az kaldıysa iptal edilemez.)
- Bu nesneyle ilgili hangi olaylar meydana gelir? (ör. **AppointmentBooked**, **AppointmentCancelled**, **ReminderScheduled**).

Bu sayede, her resource'un sadece bir *veri torbası* değil, belirli kuralları ve davranışları olan bir domain nesnesi olduğu netleşir.

İlişkileri ve Kardinaliteleri Çıkarmak

Ardından domain kavramları arasındaki ilişkileri düşünürüz:

- Bir **Patient**'in birden fazla **Appointment**'i olabilir mi? (genellikle evet, 1–N).
- Bir **Doctor**'un birden fazla **Appointment**'i var mıdır?
- **TimeSlot** başlı başına bir entity midir, yoksa sadece **startAt**/**endAt** alanlarıyla temsil edilen bir değer midir?

Bu ilişkiler, daha sonra veritabanı şeması ve API resource yapısına doğrudan yansıtılacaktır.

Domain Modelinden Resource Tasarımına Geçiş

Artık elimizde:

- Domain kavramları (ör. **Patient**, **Doctor**, **Appointment**, **Reminder**),
- Bu kavramların alanları ve iş kuralları,
- Aralarındaki ilişkiler bulunmaktadır.

Bu bilgileri kullanarak REST resource'ları tanımlarız. Örneğin:

```
GET    /patients
GET    /patients/{id}
GET    /patients/{id}/appointments
```

```
POST   /appointments
GET    /appointments/{id}
PATCH /appointments/{id}
POST   /appointments/{id}/cancel
```

Burada dikkat edilmesi gereken nokta şudur: **patients** ve **appointments** gibi resource isimleri, doğrudan veritabanı tablolarından türetilmek yerine, önce tanımlanan domain kavramlarından üretilmiştir.

Table-First ve Domain-First Yaklaşımlarının Karşılaştırılması

Table-first yaklaşımında tasarım genellikle şu şekilde başlar:

- “Bir **users** tablosu açalım, bir **appointments** tablosu açalım, araya foreign key koyalım.”
- İş kuralları sonradan ve dağınık şekilde koda serpiştirilir.

Domain-first yaklaşımda ise sıralama tersine çevrilir:

1. Önce iş biriminin ihtiyaçları ve domain kuralları netleştirilir.
2. Kullanım senaryolarından domain kavramları ve ilişkiler çıkarılır.
3. Bu kavramlar etrafında tutarlı bir domain modeli oluşturulur.
4. En son aşamada, veritabanı şeması ve API resource tasarımı bu modele göre şekillenir.

Bu sayede API, sadece mevcut tablo yapısını yansıtan bir katman olmaktan çıkar; iş kurallarını ve domain mantığını açık ve anlaşılır bir şekilde temsil eden, bakımı ve evrimi daha kolay bir arayüze dönüşür.

DDD'nin Katmanlara Yayılması: BE, DB ve FE

Domain Driven Design (DDD), belirli bir teknolojiye değil, **ortak bir domain diline** ve bu dili yansıtan **tutarlı modellere** odaklanır. Bu nedenle domain odaklı düşünme yalnızca backend katmanında değil, veritabanı (DB) ve frontend (FE) tarafında da kendini göstermelidir.

Backend (Domain Katmanı)

- *Entity, Value Object, Aggregate, Domain Service* gibi kavramlar backend'de merkezî rol oynar.
- Örnek domain tipleri: **User, Appointment, Subscription, Payment**.
- İş kuralları, dağınık **if** blokları yerine bu domain nesnelerinin içinde ifade edilir (**Subscription.cancel()**, **Appointment.book()** vb.).

Veritabanı (DB Şeması)

- DDD, her aggregate için bire bir tablo zorunluluğu getirmez; ancak:
 - Tablo ve kolon isimlerinin domain dilini yansıtmaması (**users, appointments, subscriptions** vb.),
 - İlişkilerin domain'de tanımlanan ilişkilere (ör. **User–Appointment** 1–N) karşılık gelmesi beklenir.
- Veritabanı, domain modeline sadık kalarak performans ve raporlama için optimize edilebilir (denormalize tablolar, **read_model_*** tabloları vb.).
- Önemli olan, backend domain modelinin bu şemaya makul bir şekilde map edilebilmesidir.

Frontend (FE ve İstemci Tarafı)

- Frontend de yalnızca “butonlar”dan ibaret değildir; orada da **User, Appointment, Subscription** gibi domain tiplerini temsil eden modeller bulunur.
- İyi bir pratik, API'den gelen ham veri (DTO) ile frontend'in kullandığı domain modeli arasında ayrım yapmaktır:
 - **AppointmentDto** (API cevabı),
 - **Appointment** veya **AppointmentViewModel** (UI'da kullanılan domain modeli).
- İsimlendirme ve alanlar, backend'deki domain diline mümkün olduğunca paralel tutulmalıdır (örneğin **SubscriptionStatus.ACTIVE** kavramı FE'de de aynı anlamla kullanılmalıdır).
- Ekranlar ve state yönetimi de domain'e göre gruplanır (örneğin **appointments, subscriptions, billing** gibi feature/domain odaklı modüller).

Sonuç

- DDD'nin özü, **ubiquitous language** (herkesin paylaştığı ortak domain dili) oluşturmaktır.
- Bu dilin yalnızca backend kodunda değil, veritabanı şemasında ve frontend modellerinde de tutarlı şekilde kullanılması, sistemin bakımını, iletişimini ve evrimini önemli ölçüde kolaylaştırır.
- Her katmanda aynı “tasarım deseni setini” bire bir kopyalamak şart değildir; ancak farklı katmanlarda aynı kavram için farklı isimler ve anlamlar kullanmak, DDD'nin ruhuna terstir.

Listeleme, Filtreleme ve Sıralama

Liste endpoint'leri tipik olarak pagination ve filtreleme parametreleri alır:

- Limit / offset:
 - GET /users?limit=20&offset=40
- Cursor bazlı:
 - GET /users?cursor=abc123
- Filtreler:
 - GET /orders?status=paid&from=2025-01-01&to=2025-01-31
- Sıralama:
 - GET /events?sort=-createdAt (eksi: azalan)

Limit/Offset ve Cursor Tabanlı Sayfalama Arasındaki Fark

Sayfalama için iki yaygın yaklaşım vardır: **limit/offset** ve **cursor** tabanlı sayfalama.

Limit / Offset

- Örnek istek: GET /users?limit=20&offset=40
- Anlamı: İlk 40 kaydı atla, sonraki 20 kaydı getir.
- Genellikle SQL'de LIMIT 20 OFFSET 40 ile uygulanır.

Avantajları:

- Basit ve anlaşılması kolaydır.
- “Sayfa 1, sayfa 2, sayfa 3” gibi klasik pagination arayüzleri için uygundur.

Dezavantajları:

- **Performans:** OFFSET değeri büyüdükçe veritabanı önce çok sayıda satırı atlamak zorunda kalır; bu da yavaşlamaya neden olabilir.
- **Tutarlılık:** Kayıtlar araya eklenip silindikçe, **offset** sabit kalsa bile hangi satırların geleceği değişebilir (sayfalar “kayabilir”).

Cursor Tabanlı Sayfalama

- Örnek istekler:
 - İlk sayfa: GET /users?limit=20
 - Sonraki sayfa: GET /users?limit=20&cursor=abc123
- Sunucu, ilk sayfayı dönerken bir **nextCursor** üretir:

```
1 {  
2   "data": [ /* ilk 20 kayıt */ ],  
3   "nextCursor": "abc123"  
4 }
```

İstemci, bir sonraki sayfayı isterken bu cursor değerini geri gönderir. Cursor genellikle son döndürülen kaydın kimliğini veya sıralama anahtarını (örneğin id ya da **createdAt**) temsil eder.

Avantajları:

- **Daha iyi performans:** Sorgular genellikle WHERE id > :lastId ORDER BY id LIMIT :limit gibi indeks dostu şekilde yazılabilir.

- **Daha tutarlı sonuçlar:** “Bu kayıttan sonrasını getir” mantığı olduğu için, araya yeni kayıtlar eklense bile sayfalar daha öngörülebilir davranır. Özellikle timeline, feed, log listeleri gibi sürekli büyüyen verilerde uygundur.
- Sonsuz scroll (*infinite scroll*) gibi “daha fazlasını yükle” desenleri için idealdir.

Dezavantajları:

- Uygulaması **limit/offset** kadar sezgisel değildir; cursor değerinin üretilmesi ve çözümlenmesi gerekir.
- “Doğrudan sayfa 5’e git” gibi rastgele sayfa atlama senaryoları için uygun değildir; genellikle sırayla ileri (ve bazen geri) gezinmeye odaklanır.

Özet

- Küçük veri setleri ve basit yönetim ekranlarında **limit/offset** genellikle yeterlidir.
- Büyük tablolar, yüksek trafik ve sürekli yeni kayıt eklenen senaryolarda (örneğin etkinlik logları, sosyal medya akışları, işlem listeleri) **cursor tabanlı sayfalama** daha performanslı ve daha tutarlı sonuçlar üretir.

Request ve Response Sözleşmeleri

Bu bölümde, istemci (frontend veya başka servisler) ile backend arasındaki veri alışverişinin şeklini belirleyen *request* ve *response* sözleşmelerini ele alıyoruz. Amaç, hem insanlar hem makineler için **tahmin edilebilir**, **tutarlı** ve **loglanabilir** bir protokol tanımlamaktır.

Response Şekli

Bir API'nin “başarılı” cevapları için iki yaygın yaklaşım vardır:

- Doğrudan sadece resource'u dönmek,
- Resource'u bir “zarf” (*envelope*) içinde, ek meta verilerle birlikte dönmek.

Doğrudan Kaynak

En basit yaklaşım, doğrudan ilgili kaynağın JSON temsiliyle yanıt dönmektir:

```
1 {  
2   "id": 1,  
3   "name": "Alice",  
4   "status": "active"  
5 }
```

Avantajları:

- Son derece basit ve okunabilirdir.
- Overhead minimaldir; ek bir **data** veya **meta** katmanı yoktur.

Dezavantajları:

- Ek meta bilgi (örneğin **requestId**, toplam kayıt sayısı, sayfalama bilgileri) eklemek istediğinizde, bu alanlar doğrudan resource ile karışabilir.
- Farklı endpoint'lerin farklı şekillerde meta bilgi gömmesi durumunda tutarlılık zorlaşabilir.

Envelope (Zarf) Modeli

Bu yaklaşımda, asıl veri **data** alanı içinde tutulur; ek teknik veya işlevsel meta bilgiler ise **meta** gibi ayrı bir alanda taşınır:

```
1 {  
2   "data": {  
3     "id": 1,  
4     "name": "Alice",  
5     "status": "active"  
6   },  
7   "meta": {  
8     "requestId": "abc-123",  
9     "timestamp": "2025-12-03T12:34:56Z"  
10  }  
11 }
```

Liste endpoint'leri için tipik bir örnek:

```
1 {  
2   "data": [  
3     { "id": 1, "name": "Alice" },  
4     { "id": 2, "name": "Bob" }  
5   ],  
6   "meta": {  
7     "total": 127,  
8   }  
9 }
```

```
8   "limit": 20,  
9   "offset": 40,  
10  "requestId": "abc-123",  
11  "hasNextPage": true  
12 }  
13 }
```

Burada:

- **data**: Asıl iş verisini (resource'ları) içerir.
- **meta**: Sayfalama, toplam kayıt sayısı, istek kimliği, sunucu zamanı vb. ek bilgileri içerir.

Avantajları:

- Tüm endpoint'ler için **aynı iskelet** (**data** + **meta**) kullanılır; bu da frontend tarafında generic handler'lar yazmayı kolaylaştırır.
- İş verisi ile teknik/lojistik meta veriler (requestId, sayfalama, vb.) birbirine karışmaz.
- İleride yeni meta alanları eklemek (örneğin "warnings": [...]) görece daha kolaydır.

Dezavantajları:

- Tek bir resource dönen çok basit endpoint'ler için biraz daha "gösterişli" görünebilir.
- JSON yapısı bir seviye daha derin olduğundan, istemci tarafında **response.data** kullanma alışkanlığı gerektirir.

Hangi Yaklaşımı Ne Zaman Tercih Etmeli?

- Eğer API'niz temelde **küçük** ve **çok az sayfalama** / **meta bilgi** gerektiriyorsa, doğrudan kaynak dönen basit yapı yeterli olabilir.
- Eğer:
 - Birden çok liste endpoint'iniz varsa,
 - Sayfalama (**total**, **limit**, **cursor**) bilgisi taşımak istiyorsanız,
 - **requestId**, **warnings**, **deprecated** gibi ek meta veriler kullanacaksanız,o zaman **envelope (zarf) modeli** çoğu zaman daha sürdürülebilir bir seçimdir.

Standart Alanlar ve İsimlendirme

Ne tür response şekli seçilirse seçilsin, aşağıdaki türden alanları tutarlı kullanmak faydalıdır:

- **Kimlikler**: **id**, **userId**, **orderId** gibi isimler açık ve tutarlı olmalıdır.
- **Zaman damgaları**: **createdAt**, **updatedAt**, **deletedAt** gibi ISO-8601 formatında datetime alanları.
- **Durum alanları**: **status** (ör. "active", "pending", "canceled") gibi enum'a karşılık gelen string alanlar.
- **Sayfalama**: **total**, **limit**, **offset** veya **nextCursor**, **hasNextPage** gibi alanlar.

İsimlendirme standartlarının API genelinde aynı kalması, frontend ve diğer tüketicilerin öğrenme maliyetini ciddi şekilde azaltır.

Request Şekli

Response kadar, request'lerin de tutarlı olması önemlidir. Genel olarak:

- **Path parametreleri:** Kaynağın kimliğini ve hiyerarşisini ifade eder (örneğin `/users/{userId}/orders/{orderId}`).
- **Query parametreleri:** Filtreleme, sıralama, sayfalama gibi *okuma amaçlı* parametreler için kullanılır (örneğin `?status=active&limit=20`).
- **Request body:** Kaynak oluşturma / güncelleme gibi durumlarda iş verisini taşır (örneğin `POST /users` ile gelen JSON gövdesi).

Bir kaynak oluşturma isteği örneği:

```
1 POST /users
2 Content-Type: application/json
3
4 {
5   "email": "alice@example.com",
6   "name": "Alice",
7   "preferredLanguage": "tr"
8 }
```

İyi bir pratik:

- Sunucu, veritabanı ID'sini istemez; ID üretimi sunucu tarafında yapılır (id body'de değil, response'da döner).
- Güncelleme isteklerinde (PATCH/PUT), sadece güncellenebilir alanlar body'de bulunur; id yine path içinde taşınır (`/users/{id}`).

Hata Formatı

Tutarlı bir hata şeması, hem frontend hem de log analizi için kritiktir. Hata durumunda dönen JSON'un iskeleti her endpoint'te aynı olmalıdır. Basit bir örnek:

```
1 {
2   "error": {
3     "code": "VALIDATION_ERROR",
4     "message": "Input is invalid",
5     "details": [
6       { "field": "email", "message": "Invalid email format" },
7       { "field": "password", "message": "Password is too short" }
8     ]
9   }
10 }
```

error.code ve HTTP Status İlişkisi

- `error.code`, domain seviyesinde anlamlı bir sabit olmalıdır:
 - `"USER_NOT_FOUND"`
 - `"EMAIL_ALREADY_IN_USE"`
 - `"INSUFFICIENT_BALANCE"`
 - `"RATE_LIMIT_EXCEEDED"`
- HTTP status kodu ise hatanın *genel sınıfını* belirtir:
 - 400 / 422: İstemci hataları, validasyon / semantik hatalar,
 - 401 / 403: Kimlik doğrulama / yetki sorunları,
 - 404: Kaynak bulunamadı,
 - 409: Çakışma (conflict),

- 500+: Sunucu tarafı beklenmeyen hatalar.

Böylece:

- Frontend, HTTP koduna göre genel akışı (örneğin tekrar dene mi, login ekranına mı gönder) belirleyebilir.
- `error.code` üzerinden de kullanıcıya gösterilecek mesajı veya spesifik UI davranışını seçebilir.

details Alanı

`details` dizisi, özellikle form validasyonu gibi durumlarda hangi alanlarda hangi sorunun olduğunu detaylandırmak için kullanılır:

- **field:** Hatalı alanın ismi veya yolu (ör. `"email"`, `"address.postCode"`).
- **message:** Kullanıcıya gösterilebilecek veya lokalizasyon sistemine gönderilebilecek hata mesajı.

Örneğin:

```
1 {
2   "error": {
3     "code": "VALIDATION_ERROR",
4     "message": "One or more fields are invalid.",
5     "details": [
6       {
7         "field": "email",
8         "message": "This email address is already registered."
9       },
10      {
11        "field": "password",
12        "message": "Password must be at least 8 characters."
13      }
14    ]
15  }
16 }
```

Frontend tarafı, bu alanları kullanarak ilgili input bileşenlerini kırmızıya boyayabilir, altına hata mesajı yazabilir veya toplu bir uyarı gösterebilir.

Teknik Hatalar ve Loglama Açısından Öneriler

- Hata response'unda, kullanıcıya *gerekli olmayan* teknik detaylar (stack trace, iç sistem URL'leri, veritabanı hata mesajları vb.) asla doğrudan döndürülmemelidir.
- Bunun yerine:
 - Loglarda ayrıntılı teknik bilgi,
 - Response içinde ise kullanıcıya uygun, sade bir açıklama ve bir `requestId` veya `errorId` alanı bulundurulabilir.
- Örneğin:

```
1 {
2   "error": {
3     "code": "INTERNAL_ERROR",
4     "message": "An unexpected error occurred. Please try again later.",
5     "errorId": "err-20251203-xyz789"
6   }
7 }
```

Bu durumda:

- Kullanıcıya çok teknik detay vermeden “bir hata oluştu” mesajı gösterilir.
- Destek ekibi veya geliştiriciler, loglarda `errorId` veya `requestId` ile ilgili kaydı bularak inceleme yapabilir.

Özet

- Başarılı response’lar için **tutarlı bir iskelet** (doğrudan resource veya `data+meta`) belirlenmelidir.
- Hata response’ları için `error.code`, `message` ve isteğe bağlı `details` / `errorId` gibi alanları içeren **tek bir standart şema** kullanılmalıdır.
- HTTP status kodu, hatanın genel türünü; `error.code` ise domain’e özgü ayrıntılı sınıflandırmasını temsil etmelidir.

Validasyon ve Hata Yönetimi

Schema Bazlı Validasyon

Request body, query parametreleri ve path parametreleri için şema temelli validasyon kullanmak en sağlıklı yaklaşımlardan biridir. Bu amaçla, farklı ekosistemler için çeşitli kütüphaneler mevcuttur.

Node.js Ekosisteminde Yaygın Kütüphaneler

- **Zod** – TypeScript ile sık kullanılan, tip tanımı ve runtime validasyonu birleştiren modern bir şema kütüphanesi.
- **Joi** (ve **@hapi/joi**) – Yıllardır kullanılan, beyan edici (declarative) şema tanımları sunan köklü bir validator.
- **Yup** – Özellikle frontend form validasyonunda popüler, ancak Node.js backend tarafında da kullanılabilen bir şema kütüphanesi.
- **Ajv** – JSON Schema tabanlı, yüksek performanslı bir validator; büyük JSON payload'ları ve OpenAPI/Swagger şemaları için yaygın olarak tercih edilir.
- **express-validator** – Express.js için route/middleware seviyesinde request validasyonu sağlayan pratik bir araç.
- **celebrate** – Joi şemalarını Express router üzerine kolayca entegre etmeye yarayan bir middleware katmanı.
- **class-validator** (+ **class-transformer**) – Özellikle NestJS ekosisteminde DTO sınıfları üzerinde dekoratörlerle validasyon tanımlamak için kullanılan kombinasyon.
- **io-ts**, **runtypes**, **superstruct**, **@sinclair/typebox** – Tip güvenliği ve runtime validasyonu birleştirmek isteyen, daha gelişmiş veya fonksiyonel programlama yaklaşımını benimseyen projelerde tercih edilen alternatifler.

Genel prensip, seçilen kütüphaneden bağımsız olarak, **tüm dış girdilerin** (**req.body**, **req.query**, **req.params**, header'lar, hatta environment değişkenleri) için açık ve tekrar kullanılabilir şemalar tanımlamak ve bu şemaları merkezi bir validasyon katmanı üzerinden çalıştırmaktır.

Validasyon aşaması:

1. Request alınır.
2. Şema ile doğrulanır.
3. Hatalı ise standart hata formatıyla 400 veya 422 döndürülür.
4. Doğruysa controller / service katmanına geçilir.

Kimlik Doğrulama ve Yetkilendirme

Authentication (Auth)

Kimlik doğrulama modeli net olmalıdır:

- JWT tabanlı,
- Session tabanlı,
- OAuth2 / OpenID Connect,
- API key tabanlı (servis-servis iletişimde).

Tipik auth endpoint'leri:

- POST /auth/login
- POST /auth/refresh
- POST /auth/logout
- GET /me (mevcut kullanıcının profilini getirir)

Authorization (AuthZ)

Yetkilendirme rol veya policy tabanlı olabilir:

- **Role-based:** admin, user, premium_user vb.
- **Attribute-based (ABAC):** Kullanıcı özellikleri + resource özellikleri + context.

Her korunan endpoint:

- **Authorization:** Bearer <token> header'ı bekler.
- Token doğrulandıktan sonra, kullanıcı kimliği ve rolleri request context'ine konur.

Performans: Pagination, Cache ve N+1 Problemi

Pagination Zorunluluğu

Üretim ortamında, “tüm kayıtlar” listesini dönen endpoint’lerden kaçınılmalıdır. Her liste endpoint’i:

- `limit` ve `offset` parametreleri,
- veya `cursor` tabanlı pagination kullanılmalıdır.

Cache Stratejileri

Cache, hem yanıt sürelerini düşürmek hem de arka uç sistemlerine binen yükü azaltmak için kullanılan temel bir optimizasyon tekniğidir. Genel olarak iki ana katmandan söz edebiliriz:

- **HTTP seviyesinde cache** (tarayıcı, CDN, reverse proxy),
- **Sunucu tarafı cache** (uygulama içi in-memory cache, Redis vb.).

HTTP Seviyesinde Cache

HTTP protokolü, istemci ve ara katmanların (tarayıcı, CDN, reverse proxy) yanıtları cache’leyebilmesi için yerleşik mekanizmalar sağlar.

Cache-Control Header’ı `Cache-Control` başlığı, yanıtın nasıl ve ne kadar süre cache’lenebileceğini belirtir. Örnekler:

- `Cache-Control: public, max-age=300`
 - Yanıt hem tarayıcı hem de ara cache’ler (CDN, proxy) tarafından 300 saniye boyunca saklanabilir.
- `Cache-Control: private, max-age=60`
 - Yanıt yalnızca *tek bir kullanıcıya* özel kabul edilir (örneğin kullanıcıya özel dashboard) ve tarayıcı tarafında 60 saniye cache’lenebilir; paylaşılan proxy’ler tarafından cache’lenmemelidir.
- `Cache-Control: no-store`
 - Yanıt hiçbir yerde (ne tarayıcıda ne proxy’de) saklanmamalıdır. Özellikle hassas veriler (şifre, token, kişisel veriler) için kullanılır.
- `Cache-Control: no-cache`
 - Yanıt cache’te saklanabilir, ancak her istek öncesinde sunucuya doğrulama (revalidation) yapılmalıdır.

Diğer İlgili Header’lar

- `Last-Modified / If-Modified-Since:`
 - Kaynağın son değiştirilme zamanını bildirir ve istemci bu değeri kullanarak “bu tarihten beri değişti mi?” diye sorabilir.
- `Expires:`
 - Yanıtın geçerliliğinin biteceği mutlak zamanı belirtir (genellikle `Cache-Control` yerine daha az tercih edilir).

Sunucu Tarafında Cache

Uygulama sunucusu tarafında, sık kullanılan veya üretilmesi pahalı olan veriler için *sunucu içi* cache mekanizmaları kullanılabilir.

In-Memory Cache

- Uygulama prosesinin belleğinde tutulan basit cache yapıları (Map, LRU cache kütüphaneleri vb.).
- Örneğin:
 - Sık sorgulanan konfigürasyon değerleri,
 - Değişmeyen referans veriler (ülke listesi, para birimleri vb.).

Dikkat edilmesi gereken noktalar:

- **Çoklu instance:** Uygulama birden fazla instance olarak çalışıyorsa, her instance kendi in-memory cache'ine sahip olur. Bu durumda cache tutarlılığını sağlamak zorlaşabilir.
- **Bellek kullanımı:** Yanlış yapılandırılmış bir cache, süreç içinde bellek tüketimini aşırı artırabilir. LRU gibi *eviction policy* (silme stratejileri) kullanmak önemlidir.
- **Process restart:** Uygulama yeniden başlatıldığında in-memory cache tamamen kaybolur; bu durum kabul edilebilir mi, iş gereksinimlerine göre değerlendirilmelidir.

Dağıtık Cache: Redis vb.

- **Redis**, Memcached gibi sistemler, birden fazla uygulama instance'ı tarafından paylaşılan, yüksek performanslı bir key-value store sağlar.
- Kullanım örnekleri:
 - Sık kullanılan sorgu sonuçlarının cache'lenmesi (örneğin “son 10 popüler ürün”),
 - Kullanıcı oturum bilgileri veya feature flag'ler,
 - Rate limit sayaçları (örneğin “bu IP son 1 dakikada kaç istek attı?” bilgisi).

Redis gibi bir cache kullanırken:

- **TTL (time-to-live)** sürelerini doğru belirlemek önemlidir:
 - Çok kısa TTL → yeterli performans kazancı olmayabilir.
 - Çok uzun TTL → bayat (stale) veri riski artar.
- **Cache key tasarımı** önemlidir:
 - Örnek: "user:123:profile", "product:42:summary".
 - Filtre ve sayfalama içeren sonuçlar için: "list:products:category=shoes&page=2" gibi deterministik anahtarlar.
- **Invalidasyon stratejisi** net olmalıdır:
 - “*Cache-aside*” (lazy loading) deseni:
 1. Önce cache'e bak, yoksa veritabanından oku.
 2. Veritabanından okunan sonucu cache'e yaz.
 3. Veri değiştiğinde ilgili cache anahtarını sil.
 - “*Write-through*” veya “*write-behind*” gibi daha gelişmiş stratejiler, yüksek trafikli sistemlerde düşünülebilir.

Cache Kullanırken Dikkat Edilmesi Gerekenler

- **Tutarlılık vs Performans:**
 - Çok agresif cache, kullanıcılara bayat veri gösterebilir.
 - Hiç cache kullanmamak ise gereksiz yük ve yüksek gecikme yaratabilir.
- **Kullanıcıya Özel Veriler:**
 - Kullanıcıya özgü yanıtlar (örneğin /me endpoint'i) için **Cache-Control: private** veya **no-store** tercih edilmelidir.
 - Paylaşılan cache'lerde (CDN, reverse proxy) kullanıcıya özel verilerin yanlışlıkla paylaşılmasına özen gösterilmelidir.
- **İzleme (Monitoring):**
 - Cache *hit/miss oranları*, ortalama yanıt süresi, memory kullanımı gibi metrikler izlenmelidir.
 - Bu metrikler, cache stratejisinin gerçekten işe yarayıp yaramadığını değerlendirmek için kritik önemdedir.

N+1 Sorgu Problemi

ORM'ler kullanılırken (Sequelize, TypeORM, Prisma vb.), ilişki yükleme sırasında N+1 sorgu problemlerine dikkat edilmelidir. Çözümler:

- Join kullanmak,
- Batch loading (örneğin GraphQL'de DataLoader),
- Read model / özel DTO ile tek seferde gerekli alanları getirmek.

Güvenlik Temelleri

Güvenlik, bir API tasarımında *en sona eklenen bir özellik* değil, en baştan itibaren gözetilmesi gereken bir mimari kısıttır. Bu bölümde, özellikle OWASP bakış açısıyla temel prensipler ve çok kiracılı (multi-tenant) sistemlerde dikkat edilmesi gereken noktalar ele alınmaktadır.

OWASP Perspektifi

OWASP (Open Web Application Security Project), hem web uygulamaları hem de API'ler için en sık rastlanan güvenlik açıklarını listeleyen ve bunlara karşı alınabilecek önlemleri belirleyen bir topluluktur. API güvenliği için en azından şu temel prensipler uygulanmalıdır:

Girdi Doğrulama ve Sanitizasyon

- Tüm dış girdi kaynakları (HTTP body, query parametreleri, path parametreleri, header'lar, cookie'ler, environment değişkenleri vb.) **şema bazlı** validasyon katmanından geçirilmelidir.
- Beklenen tip, aralık, uzunluk, format gibi kurallar açıkça tanımlanmalı ve bunlara uymayan veriler reddedilmelidir.
- Metin tabanlı girişlerde (özellikle HTML çıktısına giden değerlerde) gerekiyorsa *output encoding* ve XSS'e karşı koruma mekanizmaları uygulanmalıdır.

SQL/NoSQL Injection'a Karşı Korunma

- Tüm veritabanı sorgularında **parametrik sorgular** (prepared statement) kullanılmalı, string birleştirme ile dinamik SQL oluşturulmamalıdır.
- ORM kullanılıyorsa dahi, **where** koşulları ve ham SQL bölümlerinde kullanıcı girdisi doğrudan gömülmemeli, her zaman parametreler üzerinden geçirilmelidir.
- NoSQL veritabanlarında (MongoDB vb.) da benzer şekilde, filtre objelerine kullanıcı girdisi eklenirken beklenmeyen operatör enjeksiyonlarına karşı dikkatli olunmalıdır (örneğin \$where, \$ne vb.).

Güvenli İletişim: HTTPS Zorunluluğu

- Production ortamında tüm trafik **TLS** (HTTPS) üzerinden akmalıdır.
- HTTP istekleri sunucu tarafında otomatik olarak HTTPS'e yönlendirilmelidir (örneğin HTTP 301/302 ile).
- **Strict-Transport-Security** (HSTS) header'ı kullanılarak tarayıcılara her zaman HTTPS kullanmaları gerektiği bildirilebilir.

Kimlik Doğrulama ve Yetkilendirme Zafiyetleri

- Kimlik doğrulama (authentication) mekanizması güçlü şifre politikaları, çok faktörlü doğrulama (MFA) gibi ek önlemlerle desteklenmelidir.
- Access token ve refresh token'lar güvenli şekilde üretilmeli, saklanmalı ve uygun süre sonunda geçersiz kılınmalıdır.
- Her **korunan endpoint** için, sadece kullanıcının kimliği değil, ilgili kaynağa erişim yetkisi (authorization) de kontrol edilmelidir:
 - Örneğin: `/orders/{id}` endpoint'ine istek geldiğinde, o `id`'li siparişin gerçekten ilgili kullanıcıya ait olup olmadığı mutlaka doğrulanmalıdır.
- Rol tabanlı (RBAC) veya politika tabanlı (ABAC) modeller kullanılırken, rollere çok geniş yetkiler verilmemeli, *en az ayrıcalık (least privilege)* ilkesi gözetilmelidir.

Rate Limiting ve Kötüye Kullanımın Önlenmesi

- Login, signup, şifre sıfırlama, e-posta doğrulama gibi hassas endpoint'ler için IP veya kullanıcı bazlı **rate limiting** uygulanmalıdır.
- API genelinde, DDoS benzeri saldırılara ve brute-force denemelere karşı, istek sayısını kullanıcı, IP veya access token bazında sınırlayan mekanizmalar kullanılmalıdır.
- Rate limit ihlali durumunda genellikle `429 Too Many Requests` kodu ile yanıt verilir ve istek sıklığını azaltması yönünde bilgilendirici bir mesaj döndürülür.

Gizli Bilgilerin Yönetimi (Secrets Management)

- Secret bilgiler (API anahtarları, veritabanı kullanıcı adı/parolası, JWT imzalama anahtarları, OAuth client secret'ları vb.) **asla** kaynak kod deposuna düz metin olarak gömülmemelidir.
- Bu tür bilgiler:
 - Environment değişkenleri aracılığıyla,
 - Veya bir *secret manager* (örn. bulut sağlayıcının sunduğu gizli anahtar kasaları) üzerinden yönetilmelidir.
- Loglama yapılırken, secret değerlerinin veya hassas kişisel verilerin (kişisel tanımlayıcı bilgiler, şifreler vb.) log'lara yazılmamasına dikkat edilmelidir.

Güvenlik Başlıkları ve CORS

- **Security headers:** Content-Security-Policy, X-Content-Type-Options, X-Frame-Options, Referrer-Policy gibi HTTP başlıkları, özellikle tarayıcı tabanlı istemciler için ek koruma sağlar.
- **CORS (Cross-Origin Resource Sharing)** yapılandırılırken:
 - '*' ile her kökene izin vermek yerine, sadece güvenilen domain'ler açıkça listelenmelidir.
 - Kimlik bilgisi gerektiren isteklerde (`withCredentials`) CORS kuralları ayrıca gözden geçirilmelidir.

Güncellemeler, Bağımlılıklar ve Güvenlik Testleri

- Üçüncü parti kütüphaneler ve framework'ler düzenli olarak güncellenmeli, bilinen güvenlik açıklarına sahip versiyonlar kullanılmamalıdır.
- Bağımlılık analizi ve zafiyet taraması (*dependency scanning*) araçları (örneğin *npm audit*, SCA araçları vb.) düzenli olarak çalıştırılmalıdır.

- Otomatik ve manuel **güvenlik testleri** (penetrasyon testleri, fuzzing, güvenlik odaklı kod incelemeleri) yazılım yaşam döngüsünün bir parçası haline getirilmelidir.

Multi-tenant API'ler

Birden fazla müşteri veya organizasyonu (*tenant*) aynı altyapı üzerinde barındıran sistemlere **multi-tenant** sistemler denir. Bu tür sistemlerde en kritik konu, her tenant'ın verisinin diğerlerinden **tamamen yalıtılmış** olmasıdır.

Tenant Kimliğinin Taşınması

Tenant bilgisinin istek boyunca taşınması için farklı stratejiler kullanılabilir:

- **Subdomain tabanlı:**
 - Örnek: `customer-a.example.com`, `customer-b.example.com`.
 - Sunucu, host header'ından tenant'ı çıkarır.
- **Path tabanlı:**
 - Örnek: `/tenants/{tenantId}/orders`.
 - Tenant kimliği URL path'i içinde taşınır.
- **Header tabanlı:**
 - Örnek: `X-Tenant-Id: tenant_123`.
 - Genellikle arka uç servisleri arası iletişimde veya API gateway ile birlikte kullanılır.
- **JWT claim'i:**
 - Kullanıcının access token'ı içinde `tenant_id` veya benzeri bir claim bulunur.
 - Uygulama, bu claim'i okuyarak hangi tenant bağlamında işlem yapacağını belirler.

Önemli prensip: **Tenant kimliğini yalnızca istemcinin keyfi olarak gönderdiği değerlere güvenerek belirlememek**, mümkün olduğunda kimlik doğrulama (JWT claim, session, SSO) ile ilişkilendirmektir.

Veri Erişiminde Tenant İzolasyonu

Her veri erişimi sırasında tenant izolasyonunu garanti etmek için:

- Tüm sorgularda **tenant filtresi** zorunlu olmalıdır:
 - Örnek: `SELECT * FROM orders WHERE tenant_id = :tenantId AND id = :orderId`.
- ORM kullanılıyorsa:
 - Global *query filter* veya *scope* mekanizmalarıyla `tenant_id` filtresinin her sorguya otomatik eklenmesi sağlanabilir.
 - Geliştiricilerin bu filtreyi “unutması”nı engelleyen soyutlamalar kullanılmalıdır.
- “*Cross-tenant data access*” (yanlış tenant verisini görmek) senaryolarını yakalamak için:
 - Hem unit/integration test'leri hem de otomatik güvenlik testleri tasarlanmalıdır.
 - Log'larda `tenant_id` bilgisi de tutulmalı, şüpheli erişimler izlenmelidir.

Tenant İzolasyon Modelleri

Multi-tenant sistemlerde kullanılan yaygın veri modeli yaklaşımları:

- **Paylaşılan şema, tenant_id kolonu:**
 - Tüm tenant'lar aynı tabloları kullanır; her satırda bir `tenant_id` alanı bulunur.

- Uygulama, her sorguda `tenant_id` filtresi uygular.
- Ölçeklenebilir ve bakımı görece kolaydır; fakat izolasyon hatası yapılırsa veri sızıntısı riski vardır.
- **Ayrı şemalar:**
 - Her tenant'ın kendi veritabanı şeması (`tenant_a.orders`, `tenant_b.orders`) bulunur.
 - İzolasyon daha güçlüdür; yönetim ve migrasyon maliyeti artar.
- **Ayrı veritabanları:**
 - Her tenant'ın tamamen ayrı bir veritabanı vardır.
 - En yüksek izolasyon seviyesi; ancak operasyonel karmaşıklık ve maliyet artar.

Seçilecek model, *güvenlik gereksinimleri*, *ölçek* ve *operasyonel kapasiteye* göre belirlenmelidir.

Günlükleme (Logging) ve Denetim (Audit) İzi

Multi-tenant yapılar, olayların hangi tenant bağlamında gerçekleştiğini bilmek kritik önemdedir:

- Her log kaydında `tenant_id`, `user_id`, `requestId` gibi alanlar tutulmalıdır.
- Hassas işlemler (örneğin rol değişiklikleri, yetki güncellemeleri, toplu silme işlemleri) için detaylı **audit log** kayıtları oluşturulmalıdır:
 - Kim yaptı?
 - Ne zaman yaptı?
 - Hangi değerleri neye çevirdi?
 - Hangi tenant üzerinde işlem yapıldı?
- Bu kayıtlar, hem güvenlik olaylarını araştırmak hem de müşteri denetimleri (compliance) için gereklidir.

Gözlemlenebilirlik: Logging, Metrics, Tracing

Structured Logging

Logların JSON formatında, makine tarafından işlenebilir olması tavsiye edilir:

- `timestamp`,
- `level` (INFO, WARN, ERROR),
- `requestId`,
- `userId`,
- `path`, `method`, `statusCode`,
- `durationMs`.

Metrikler

Temel metrikler:

- Request sayısı (QPS),
- Ortalama ve p95/p99 response süreleri,
- Hata oranları,
- Önemli iş metrikleri (yaratılan sipariş sayısı, başarılı ödeme oranı vb.).

Health Check

Sistem durumunu izlemek için:

- `GET /health` veya `/status` endpoint'i,
- Gerekirse DB, queue gibi bağımlılıkları kontrol eden alt kontroller.

API Evrimi ve Versiyonlama

Versiyonlama Stratejileri

Yaygın yaklaşım: URL bazlı:

- `/api/v1/...`
- `/api/v2/...`

Alternatif olarak, “versionless” ama çok dikkatli evrim geçiren API’ler de tasarlanabilir; bu durumda:

- Yeni alan eklemek genellikle güvenlidir.
- Alan silmek veya anlamını değiştirmek *breaking change* sayılır.

Deprecation Stratejisi

- Eski alanlar bir anda kaldırılmak yerine önce deprecated ilan edilir.
- Deprecation süresi (örneğin 6 ay) ve kaldırılma tarihi dokümantasyonda belirtilir.
- Loglar üzerinden hangi client’ların hala eski alanları kullandığı takip edilir.

Event-Driven, Webhook'lar ve Async İşler

Uzun Süreli İşlemler

Bazı işlemler (büyük rapor üretimi, video işleme, büyük veri setlerinin içe aktarılması vb.) bir HTTP isteğinin “normal” süresinden çok daha uzun sürebilir. Bu tür durumlarda, isteği *senkron* olarak sonuna kadar bekletmek yerine, işlemi arka planda çalıştırmak ve istemciye daha sonra sonucu alabileceği bir mekanizma sunmak daha sağlıklıdır.

Bu desen genel olarak şu şekilde işler:

1. İstemci uzun sürecek bir operasyonu başlatmak için bir **POST** isteği gönderir (örneğin **POST /reports**).
2. Sunucu, işlemi hemen bitirmek yerine bir *job* (görev) kaydı oluşturur ve isteği **202 Accepted** kodu ile yanıtlar.
3. Yanıtta, ilgili job kaynağının URL'si (örneğin **/jobs/{id}**) döndürülür.
4. İstemci, bu job kaynağını periyodik olarak sorgulayarak (*polling*) işlemin durumunu takip eder veya alternatif olarak sunucu, işlem bittiğinde istemciye *webhook* ile bildirim gönderir.

202 Accepted ve Job Kaynağı

Örnek bir akış:

1. İstemci rapor üretmek için istek gönderir:

```
1 POST /reports
2 Content-Type: application/json
3
4 {
5   "from": "2025-01-01",
6   "to": "2025-03-31",
7   "format": "pdf"
8 }
```

2. Sunucu, isteği aldıktan hemen sonra uzun süren işlemi arka planda başlatır ve şu yanıtı döner:

```
1 HTTP/1.1 202 Accepted
2 Location: /jobs/123
3 Content-Type: application/json
4
5 {
6   "jobId": "123",
7   "status": "pending",
8   "createdAt": "2025-12-03T10:00:00Z"
9 }
```

Burada:

- 202 Accepted, isteğin başarıyla *kabul edildiğini* fakat henüz *tamamlanmadığını* ifade eder.
 - Location: /jobs/123, bu işlemi temsil eden job kaynağının URL'sidir.
3. İstemci, işlemin durumunu sorgulamak için periyodik olarak **/jobs/123** kaynağına istek atar:

```
1 GET /jobs/123
2
3 {
4   "jobId": "123",
```

```
5   "status": "running",
6   "progress": 60
7 }
```

4. İşlem tamamlandığında job kaynağı şöyle bir yanıt dönebilir:

```
1 GET /jobs/123
2
3 {
4   "jobId": "123",
5   "status": "completed",
6   "resultUrl": "/reports/987"
7 }
```

5. İstemci, son adımda asıl sonucu (örneğin üretilen PDF raporu) **resultUrl** üzerinden alır:

```
1 GET /reports/987
```

Bu desene genellikle **job/polling** yaklaşımı denir: uzun süren işlem bir job kaynağı ile temsil edilir ve istemci sonucu almak için bu job'u düzenli aralıklarla *poll* eder.

Alternatif: Webhook ile Bildirim

Polling yaklaşımında istemci belirli aralıklarla “bitti mi?” diye sormak zorundadır. Bazı durumlarda, bunun yerine sunucunun işlem bittiğinde istemciye *kendiliğinden haber vermesi* tercih edilir. Bu senaryoda **webhook** yaklaşımı kullanılır.

Genel akış:

1. İstemci, uzun sürecek işlemi başlatırken bir **webhookUrl** bildirir:

```
1 POST /reports
2 Content-Type: application/json
3
4 {
5   "from": "2025-01-01",
6   "to": "2025-03-31",
7   "format": "pdf",
8   "webhookUrl": "https://client-app.com/report-ready"
9 }
```

2. Sunucu yine bir job oluşturur ve 202 Accepted + /jobs/{id} bilgisiyle yanıt verir.

3. İşlem tamamlandığında sunucu, istemcinin verdiği **webhookUrl**'ye bir HTTP isteği gönderir:

```
1 POST https://client-app.com/report-ready
2 Content-Type: application/json
3
4 {
5   "jobId": "123",
6   "status": "completed",
7   "reportUrl": "https://api.example.com/reports/987"
8 }
```

Bu sayede:

- İstemci sürekli olarak job kaynağını sorgulamak zorunda kalmaz.
- Sunucu, işlem tamamlandığında tek seferlik bir bildirimle süreci sonlandırabilir.

Mobil İstemciler ve BFF

Mobil, web ve admin panel gibi farklı istemciler (client) için API tasarlarlarken, tek bir “genel amaçlı” backend’in üzerine *Backend-for-Frontend (BFF)* katmanı eklemek, hem performans hem de geliştirilebilirlik açısından önemli avantajlar sağlar. Ayrıca, mobil istemcilerde *offline-first* yaklaşımlar ve senkronizasyon stratejileri ayrı bir önem taşır.

Backend-for-Frontend (BFF) Modeli

Backend-for-Frontend, her farklı istemci türü için (örneğin mobil uygulama, web uygulaması, admin panel) özel olarak tasarlanmış, istemciye yakın (yakın anlamda: API yüzü) bir aracı backend katmanıdır.

BFF’nin Rolü

- **UI’ye Özel Endpoint’ler:**
 - Mobil veya web arayüzünde ihtiyaç duyulan veriler, çoğu zaman birden fazla domain servisinden gelir.
 - BFF, bu farklı kaynaklardan gelen verileri birleştirerek (aggregation/composition) frontend’e *işe yarar tek bir DTO* olarak sunar.
 - Örnek: `GET /mobile/home-feed` endpoint’i, kullanıcı bilgisi + son bildirimler + önerilen içerikleri tek çağrıda dönebilir.
- **Domain Servislerinin Sade Kalması:**
 - Domain servisleri, olabildiğince *generic* ve *UI bağımsız* tutulur (örneğin `/users`, `/orders`, `/subscriptions` gibi).
 - BFF, bu generic endpoint’leri kullanarak UI’ye uygun kompozit modeller üretir.
- **Sunucu Taraflı Adaptasyon:**
 - Farklı cihaz türlerine göre (mobil vs masaüstü) farklı payload boyutları, farklı alan setleri sunulabilir.
 - Örneğin mobil BFF, sadece gerekli alanları içeren “hafif” JSON dönerken, web BFF daha detaylı istatistikleri de içerebilir.
- **Çapraz Kesme Kaygılar (Cross-Cutting Concerns):**
 - **Kimlik Doğrulama / Yetkilendirme adaptasyonu:** Token yenileme, session yönetimi, mobil-spesifik güvenlik önlemleri (örneğin cihaz bağlama, ek imza mekanizmaları).
 - **Feature flag / A/B test** entegrasyonu: BFF, feature flag sistemleriyle entegre olarak belirli kullanıcı segmentlerine farklı UI davranışları veya farklı veri setleri sunabilir.
 - **Ortam geçişleri ve versiyonlama:** Yeni mobil uygulama versiyonları için BFF üzerinden farklı endpoint davranışları kurgulanabilir (örneğin `/v1/mobile` vs `/v2/mobile`).

BFF Kullanmanın Avantajları

- **Daha Az Round-Trip:**
 - Mobil istemcinin bir ekranı doldurmak için 4-5 ayrı endpoint çağırması yerine, BFF bu çağrıları backend tarafında yapar ve tek bir birleşik cevap döner.
 - Bu, özellikle mobil ağ koşullarında (yüksek gecikme, düşük bant genişliği) ciddi fark yaratır.
- **UI Değişimlerinin Backend Domain’ini Bozmaması:**
 - UI tarafında ufak bir tasarım değişikliği için doğrudan domain servislerinde değişiklik yapmak gerekmez; çoğu zaman sadece BFF tarafında DTO veya mapping güncellemek yeterlidir.

- **Farklı İstemciler İçin Farklı Politikalar:**

- Örneğin, aynı domain’i kullanan bir *public API* ile *iç (internal) admin paneli* farklı hız limitlerine, farklı veri detay düzeylerine sahip olabilir.
- BFF sayesinde bu ayrımlar, domain katmanını kirletmeden sağlanabilir.

Örnek: Home Ekranı İçin BFF Olmadan ve BFF ile Örneğin bir mobil uygulamanın ana ekranında kullanıcının profili, son bildirimler, önerilen içerikler ve kısa bir istatistik özeti gösterildiğini varsayalım.

BFF olmadan, mobil istemci aşağıdaki gibi birden fazla çağrı yapmak zorunda kalabilir:

```
GET /users/me
GET /notifications?limit=5
GET /recommendations?limit=3
GET /stats/overview?range=7d
```

Bu durumda, mobil ağ üzerindeki her istek için ayrı bir HTTP round-trip gerçekleşir. Özellikle yüksek gecikmeli (örneğin 4G/5G, zayıf Wi-Fi) ortamlarda, 4-5 isteğin toplam gecikmesi kullanıcıya hissedilir bir yavaşlık olarak yansır.

BFF ile ise, mobil istemci yalnızca tek bir endpoint’e istek gönderir:

```
GET /mobile/home
```

BFF katmanı, arka planda şu çağrıları kendi içinde yapar:

```
GET /users/me
GET /notifications?limit=5
GET /recommendations?limit=3
GET /stats/overview?range=7d
```

ve bunların sonuçlarını tek bir birleşik JSON yanıtında döndürür:

```
1 {
2   "user": { "...": "..." },
3   "notifications": [ /* ... */ ],
4   "recommendations": [ /* ... */ ],
5   "stats": { /* ... */ }
6 }
```

Böylece mobil istemci için yalnızca *bir* HTTP round-trip yeterli olur; diğer tüm kompozisyon işlemleri, gecikmenin çok daha düşük olduğu backend ağı içinde gerçekleşir.

UI Değişikliği Örneği Örneğin, `/users/{id}` domain endpoint’i aşağıdaki gibi zengin bir kullanıcı nesnesi döndürsün:

```
1 {
2   "id": 123,
3   "email": "alice@example.com",
4   "firstName": "Alice",
5   "lastName": "Wonderland",
6   "avatarUrl": "https://cdn.example.com/avatars/123.png",
7   "createdAt": "2024-01-10T12:00:00Z",
8   "lastLoginAt": "2025-12-01T09:00:00Z"
9 }
```

Mobil BFF tarafında tanımlanan `GET /mobile/profile` endpoint’i ise, UI’nin ihtiyaç duyduğu alanlara göre bu veriyi aşağıdaki gibi map eder:

```

1 {
2   "firstName": "Alice",
3   "lastName": "Wonderland",
4   "avatarUrl": "https://cdn.example.com/avatars/123.png",
5   "memberSince": "2024-01-10",
6   "lastLoginAt": "2025-12-01T09:00:00Z"
7 }

```

Profil kartı tasarımı değişip, örneğin ad ve soyadın ayrı gösterilmesi veya ek bir alanın (örneğin `lastLoginAt`) eklenmesi gerektiğinde, `/users/{id}` domain endpoint'ini değiştirmeye gerek yoktur; sadece BFF katmanındaki DTO/mapping güncellenir.

Farklı İstemciler İçin Farklı Politikalar Örneği Aynı `/internal/orders/{id}` domain endpoint'ini kullanan iki farklı BFF düşünelim:

- **Public API BFF – GET `/api/orders/{id}`:**

- Dış entegrasyonlara sadece sınırlı alanlar döner:

```

1 {
2   "id": 987,
3   "orderNumber": "ORD-2025-000987",
4   "totalPrice": 39.98,
5   "status": "PAID",
6   "createdAt": "2025-11-30T10:00:00Z"
7 }

```

- API gateway seviyesinde örneğin 100 req/minute gibi sıkı bir rate limit uygulanır.

- **Admin BFF – GET `/admin/orders/{id}`:**

- Dahili kullanıcılar için daha detaylı bilgi döner:

```

1 {
2   "id": 987,
3   "orderNumber": "ORD-2025-000987",
4   "customerName": "Alice Wonderland",
5   "customerEmail": "alice@example.com",
6   "items": [ /* ... */ ],
7   "totalPrice": 39.98,
8   "internalCost": 10.00,
9   "margin": 29.98,
10  "status": "PAID",
11  "createdAt": "2025-11-30T10:00:00Z"
12 }

```

- Sadece yetkili admin kullanıcılarına açıktır ve daha yüksek bir rate limit ile korunur.

Her iki BFF de aynı domain servisine dayanmakta, fakat hangi alanların dış dünyaya açılacağı ve hangi hız limitlerinin uygulanacağı BFF + API gateway katmanında tanımlanarak, domain katmanı sade ve UI'dan bağımsız tutulmaktadır.

Offline-First ve Senkronizasyon

Mobil uygulamalar, her zaman stabil ve hızlı bir internet bağlantısına güvenemez. **Offline-first** yaklaşımı, uygulamanın belirli işlevleri *bağlantı yokken de* çalıştırabilmesini ve bağlantı geri geldiğinde merkezi backend ile senkronizasyon yapılmasını hedefler.

Yerel Depolama ve Request Kuyruklama

- **Lokal Veri Katmanı:**

- Mobil uygulama, sunucudan aldığı verileri lokal bir veritabanında (örneğin SQLite, Room, Core Data) saklar.
- UI, mümkün olduğunca bu lokal veritabanını okur; network sadece güncelleme ve senkronizasyon için kullanılır.

- **Request Kuyruğu:**

- Kullanıcı offline iken yaptığı işlemler (yeni kayıt ekleme, beğeni gönderme, yorum yazma vb.) önce lokal bir *kuyruğa* (queue) yazılır.
- Ağ bağlantısı geldiğinde veya uygun olduğunda, bu kuyruktaki istekler sırayla sunucuya gönderilir.
- Her istek için idempotent endpoint'ler ve tekrar denenebilir (retryable) operasyonlar tasarlamak önemlidir.

- **Retry Stratejileri:**

- Basit yaklaşım: Sabit aralıklarla tüm kuyruğu yeniden denemek.
- Daha gelişmiş: Exponential backoff, maksimum deneme sayısı, kalıcı hata durumlarında kullanıcıya feedback.

Çakışma (Conflict) Çözümü Stratejileri

Offline-first senaryolarda en zor problemlerden biri, aynı kaynağın hem sunucu tarafında hem de istemci tarafında *farklı şekillerde güncellenmesi* durumunda ortaya çıkan çakışmalardır.

Temel stratejiler:

- **Last-Write-Wins (LWW):**

- En son gelen güncelleme geçerli sayılır; önceki değişiklikler üzerine yazılır.
- Uygulaması kolaydır, ancak veri kaybına yol açabilir.

- **Sunucu Öncelikli (Server Wins):**

- Sunucudaki sürüm tek doğru kabul edilir; istemcinin çakışan değişiklikleri reddedilir.
- Kritik finansal veya stok verilerinde, istemci verisinin geri planda tutulması gerekebilir.

- **İstemci Öncelikli (Client Wins):**

- İstemcinin yaptığı değişiklikler, sunucu sürümünü her durumda ezer.
- Bazı offline uygulamalar (örn. kişisel notlar) için kabul edilebilir.

- **Alan Bazlı Birleştirme (Merge):**

- Aynı kaynağın farklı alanları farklı taraflarca değişmişse, alan bazında birleştirme yapılır (örneğin açıklama metni birleştirilir, etiket listesi union alınır).
- Daha karmaşıktır, ancak veri kaybını azaltır.

- **Kullanıcıya Sor (Manual Conflict Resolution):**

- Çakışma tespit edildiğinde, her iki sürüm de UI'da gösterilir ve kullanıcıdan seçim yapması istenir.
- Örneğin: “Sunucudaki sürüm” vs “Senin değişikliğin” şeklinde iki seçenek sunulabilir.

Çakışma tespiti için genellikle:

- **Versiyon numarası** (version alanı),
- **Zaman damgası** (updatedAt),
- veya **ETag** benzeri bir sürüm belirteci kullanılır.

DTO Tasarımı: “Slim List Item” ve “Detail View”

Mobil veri tüketimini azaltmak ve performansı artırmak için, aynı resource için farklı detay seviyelerinde DTO’lar tanımlamak iyi bir pratiktir.

- **Slim List Item DTO:**

- Liste ekranlarında yalnızca gerekli özet alanlar döndürülür.
- Örnek: `GET /articles` yanıtında sadece `id`, `title`, `summary`, `thumbnailUrl`.
- Böylece ağ üzerinden taşınan veri miktarı azalır, liste daha hızlı yüklenir.

- **Detail View DTO:**

- Detay ekranı için ek alanlar (tam içerik, geçmiş, ilişkili öğeler vb.) döndürülür.
- Örnek: `GET /articles/{id}` yanıtında `content`, `attachments`, `comments` gibi alanlar da yer alır.

- **BFF ile Birlikte Kullanım:**

- BFF katmanı, domain servislerinden gelen daha büyük veya karmaşık modelleri *liste* ve *detay* için farklı DTO setlerine dönüştürebilir.
- Bu sayede mobil istemci sadece ihtiyacı olan alanları, en uygun formatta alır.

Senkronizasyon Protokolleri ve İdempotensi

- **İdempotent Endpoint’ler:**

- Özellikle tekrar tekrar gönderilebilecek isteklerin (**retry**) idempotent olması önemlidir.
- Örnek: `PUT /todos/{id}` aynı gövdeyle birden fazla çağrılrsa dahi sonuç değişmemelidir.

- **İstemci Tarafı Geçici ID’ler:**

- Offline iken oluşturulan yeni kayıtlar için istemci tarafında geçici ID’ler üretilebilir (örneğin “temp-123”).
- Sunucuya başarılı şekilde kaydedildikten sonra, sunucudan gelen gerçek id ile lokal kayıt güncellenir.

- **Değişiklik Akışı (Change Feed) veya Sync Token’lar:**

- Sunucu, “son senkronizasyondan bu yana değişen kayıtlar”ı dönebilecek endpoint’ler sağlayabilir:

```
1 GET /sync?since=2025-12-01T00:00:00Z
```

- Alternatif olarak, her senkronizasyonda döndürülen bir **syncToken** kullanılarak bir sonraki senkronizasyonda sadece yeni veya değişen kayıtlar istenebilir.

Developer Experience (DX)

Dokümantasyon

- OpenAPI/Swagger tanımı,
- Örnek `curl` komutları ve Postman koleksiyonları,
- “Quickstart” bölümü: İlk request’in 10 dakika içinde atılabilmesi.

API Style Guide

Ekip içi tutarlılık için kısa bir stil rehberi:

- İsimlendirme kuralları,
- Status kod kullanım rehberi,
- Hata formatı standardı,
- Pagination, filtering ve sorting pattern’leri.

API Tasarım Checklist'i

Bu bölüm, önceki tüm başlıkları bir *checklist* haline getirir. Amaç, bir API veya endpoint tasarlarken üzerinden hızlıca geçebileceğin bir kontrol listesi sağlamaktır.

HTTP & Resource Tasarımı

- Doğru HTTP metodları kullanılıyor (GET/POST/PUT/PATCH/DELETE).
- Status kodları anlamlı ve tutarlı seçiliyor (200/201/204/400/401/403/404/409/422/500).
- URL yapıları isim tabanlı ve resource odaklı (/users, /users/{id}).
- Liste endpoint'lerinde pagination + filtreleme + sıralama desteği var.

Request / Response Sözleşmesi

- Response formatı tutarlı (doğrudan veya data/meta envelope).
- Liste cevaplarında meta.total, limit, offset/cursor bilgileri yer alıyor.
- Validasyon hataları için standart bir error şeması kullanılıyor.

Validasyon & Hata Yönetimi

- Body, query, path parametreleri şema bazlı validate ediliyor.
- İş kuralı hataları için anlamlı error.code değerleri tanımlı.
- Global error handler, exception'ları HTTP kodu + error.code + mesaj'a map ediyor.

Auth & Authorization

- Açık ve net bir auth modeli var (JWT, session, OAuth2, vb.).
- Login, refresh ve /me endpoint'leri tanımlanmış.
- Her korunan endpoint Authorization: Bearer <token> bekliyor.
- Rol veya policy tabanlı authorization uygulanıyor.

Performans

- Büyük listeler için mutlaka pagination kullanılıyor.
- Kritik endpoint'ler için cache stratejisi mevcut.
- ORM kullanımında N+1 problemleri analiz edilmiş ve önlenmiş.

Güvenlik

- Tüm input'lar validate ve sanitize ediliyor.
- Parametrik sorgular ile SQL/NoSQL injection riski azaltılıyor.
- Production ortamda yalnızca HTTPS kullanılıyor.
- Login/signup gibi hassas endpoint'lerde rate limit mevcut.
- Secret'lar environment değişkenleri veya secret manager ile yönetiliyor.

Gözlemlenebilirlik

- Structured logging yapılıyor (JSON loglar).
- Temel metrikler toplanıyor (QPS, latency, error rate).
- /health veya benzeri bir healthcheck endpoint'i mevcut.

Evrım ve Versiyonlama

- API versiyonlama stratejisi net (ör. `/api/v1`).
- Yeni alan eklemek breaking change olarak kabul edilmiyor.
- Alan silme veya anlam değışiklikleri için deprecation süreci tanımlı.

Async & Webhook

- Uzun süren işler için async/polling veya webhook pattern'i düşünülmüş.
- Webhook'lar imzalı, tekrar denemeli ve idempotent tasarlanmış.

Sonuç ve Özet İlkeler

Bu dokümanda, modern bir backend API'sinin tasarımında dikkat edilmesi gereken temel prensipleri uçtan uca ele aldık. Amaç; yalnızca “çalışan” değil, **anlaşılır, bakımı kolay, güvenli, ölçeklenebilir ve mobil dostu** bir API mimarisi için rehber niteliğinde bir çerçeve sunmaktır.

1. Domain Odaklı Tasarım ve Resource Modeli

İyi bir API tasarımının temeli, veritabanı tablolarından değil **domain kavramlarından** yola çıkmaktır. Kullanıcı, sipariş, abonelik, randevu, ödeme gibi iş alanına özgü kavramlar:

- Net birer *resource* olarak modellenmeli,
- Her birinin sorumlulukları, alanları ve ilişkileri açık bir şekilde tanımlanmalıdır.

URL'lerin **resource odaklı** olması (örneğin `/users`, `/orders/{id}`) ve eylemlerin HTTP metodlarına (GET, POST, PATCH, DELETE) yansıtılması, API'nin hem tutarlılığını hem de öğrenilebilirliğini artırır. Path içinde fiil kullanmak yerine (`/createUser` gibi), eylemi HTTP metoduna bırakmak (POST `/users`) en iyi pratiktir.

2. HTTP Semantiği, Status Kodları ve İyi Davranan Endpoint'ler

HTTP sadece “taşıyıcı” değil, aynı zamanda güçlü bir **anlamsal katmandır**. Doğru kullanılan status kodları ve header'lar:

- Başarılı senaryoları (200, 201, 204),
- İstemci hatalarını (400, 401, 403, 404, 409, 422),
- Sunucu hatalarını (5xx)

net bir şekilde ayırt etmeyi sağlar. Uzun süren işlemler için 202 **Accepted** + *job* kaynakları ve gerekirse webhook'lar kullanmak, API'yi daha dayanıklı ve kullanıcı dostu hale getirir.

Listeleme, filtreleme, sıralama ve sayfalama için:

- Küçük ve basit senaryolarda **limit/offset**,
- Büyük veri setleri ve sonsuz scroll gibi durumlarda **cursor tabanlı sayfalama**

tercih edilmelidir.

3. Tutarlı Request/Response Sözleşmeleri ve Validasyon

API'nin kullanılabilirliği, büyük ölçüde **response formatlarının tahmin edilebilir olmasına** bağlıdır. Doğrudan resource dönen yaklaşımlar basitlik sağlarken, **data + meta** içeren *envelope* modeli, sayfalama bilgisi, **requestId**, **warnings** gibi ek meta verileri taşımayı kolaylaştırır.

Hata durumları için:

- Her zaman aynı iskeleti takip eden bir **error** nesnesi,
- **error.code** (domain odaklı sabitler),
- **details** ile alan bazlı validasyon hataları

kullanmak, hem frontend geliştirmesini hem de log analizini büyük ölçüde sadeleştirir.

Tüm dış girdilerin, seçilen bir **schema bazlı validasyon kütüphanesi** (Zod, Joi, Ajv, class-validator vb.) üzerinden geçirilmesi, güvenlik ve kalite açısından olmazsa olmazdır.

4. Cache, Performans ve Ölçeklenebilirlik

Performans ve maliyet optimizasyonu için:

- HTTP seviyesinde **Cache-Control**, **ETag**, **If-None-Match** gibi header'lar kullanılarak istemci ve ara cache'ler etkinleştirilmeli,
- Sunucu tarafında, in-memory cache veya Redis gibi dağıtık cache'lerle sık kullanılan veya üretimi pahalı veriler saklanmalıdır.

İyi tasarlanmış cache anahtarları, doğru TTL değerleri ve net bir invalidasyon stratejisi, yüksek trafikli sistemlerde belirleyici fark yaratır. Cache'in daima bir *doğruluk vs performans* dengesi olduğu unutulmamalıdır.

5. Güvenlik ve Multi-tenant Bilinci

Güvenlik, her katmanda düşünülmesi gereken yatay bir sorumluluktur:

- Girdi validasyonu, parametrik sorgular, HTTPS zorunluluğu,
- Kimlik doğrulama ve yetkilendirme (RBAC/ABAC), rate limiting,
- Secret yönetimi, güvenlik başlıkları ve CORS yapılandırması,
- Bağımlılıkların güncel tutulması ve güvenlik testleri

OWASP rehberleri doğrultusunda ele alınmalıdır.

Multi-tenant mimarilerde ise:

- **tenant_id** taşıma stratejisinin (header, JWT claim, subdomain, path) net olması,
 - Her sorguda zorunlu tenant filtresinin uygulanması,
 - Veri izolasyonunu doğrulayan testler ve audit log'lar
- sistemin güvenilirliği için kritiktir.

6. Mobil İstemciler, BFF ve Offline-First Yaklaşımlar

Farklı istemci türleri (mobil, web, admin) için **Backend-for-Frontend (BFF)** katmanı:

- UI'ye özel, kompozit endpoint'ler tanımlamayı,
- Domain servislerini sade ve tekrar kullanılabilir bırakmayı,
- Farklı istemciler için farklı rate limit, veri detay seviyesi ve politika uygulamayı

mümkün kılar.

Mobil dünyada ise **offline-first** tasarım:

- Lokal depolama ve request kuyruklama,
- Çakışma çözümü stratejileri (last-write-wins, server/client wins, merge, manual resolve),
- “Slim list item” ve “detail view” DTO'ları ile ağ kullanımını minimize etme

gibi konseptlerle birlikte ele alınmalıdır.

7. Süreç, Evrim ve Belgelenebilirlik

Son olarak, iyi bir API tasarımı:

- **Tek seferlik bir aktivite** değil, yaşayan bir varlık olarak görülmelidir.
- Versiyonlama, deprecation stratejileri ve geriye dönük uyumluluk kuralları net olmalıdır.
- Açık ve güncel dokümantasyon (OpenAPI/Swagger, örnek istek/yanıtlar, hata kodu tabloları) ekip içi iletişimi ve dış tüketicilerin entegrasyon süresini ciddi biçimde kısaltır.

Bu dokümandaki ilkeler, farklı teknoloji yığınlarına (Node.js, Java, .NET, Go, vb.) ve çeşitli iş alanlarına uyarlanabilir *temel yapı taşları* olarak düşünülmelidir. Her projede ayrıntılar değişse de; domain odaklılık, tutarlı HTTP semantiği, schema bazlı validasyon, güvenlik, cache ve BFF/online–offline dengesine gösterilen özen, uzun vadede hem teknik borcu azaltacak hem de geliştirici deneyimini belirgin biçimde iyileştirecektir.